

Culture & Outils DevOps

CI/CD · Docker · Kubernetes

Formateur : Haithem Mihoubi

Module : 2 / 3

Durée : 4 jours (28 heures)

Niveau : Mixte (débutant → intermédiaire)

Public : développeurs, ops, futurs ingénieurs DevOps

Pré-requis : bases de la ligne de commande Linux et de Git

Sommaire

Avant-propos formateur	3
Jour 1 — Culture DevOps & CI/CD	4
1.1 Origine et philosophie DevOps	4
1.2 Le cycle de vie DevOps	5
1.3 Git & Git Flow (rappels utiles)	5
1.4 Anatomie d'un pipeline CI	6
1.5 Atelier — Pipeline CI	6
Quiz — Jour 1	8
Jour 2 — Docker : conteneurisation	9
2.1 Conteneurs vs machines virtuelles	9
2.2 Concepts Docker	9
2.3 Le Dockerfile	10
2.4 Docker Compose	11
2.5 Registres d'images	12
2.6 Atelier — Conteneuriser une application	12
Quiz — Jour 2	12
Jour 3 — Introduction à Kubernetes	14
3.1 Pourquoi un orchestrateur ?	14
3.2 Architecture et objets	14
3.3 Configuration : ConfigMaps, Secrets, Namespaces	16
3.4 RBAC & sécurité	17
3.5 Atelier — Déployer sur Minikube / K3s	17
Quiz — Jour 3	18
Jour 4 — Automation & DevOps avancé	19
4.1 Pipeline CI/CD de bout en bout	19
4.2 Helm Charts	20
4.3 Observabilité : Prometheus & Grafana	20
4.4 GitOps avec Argo CD	21
4.5 Atelier final — Chaîne complète CI/CD → Docker → Kubernetes	22
Évaluation finale du Module 2	22

Avant-propos formateur

Ce module est **très pratique** : privilégiez le *live coding* et laissez les participants reproduire en direct. Le fil rouge est une petite API web (« **QuickBite API** », Node.js ou Spring Boot) que l'on construit, conteneurise, déploie, puis livre via un pipeline complet.

⚠️ Préparer les postes AVANT le jour 1

Faites installer en amont : **Git**, **Docker Desktop** (ou Docker Engine + Compose), un compte **GitHub** (ou GitLab), **kubectl**, et **Minikube** (ou **K3s/kind**). Prévoyez un plan B « cloud » (GitHub Codespaces, Killercoda, Play with Docker) si une machine pose problème.

Vérification d'environnement (à exécuter au démarrage)

```
git --version
docker --version && docker compose version
kubectl version --client
minikube version      # ou: k3s --version
```

Jour 1 — Culture DevOps & CI/CD

🎯 Objectifs du Jour 1

- Expliquer ce qu'est (et n'est pas) le DevOps et le modèle CAMS.
- Décrire le cycle de vie DevOps et la place de l'automatisation.
- Maîtriser un flux Git collaboratif (branches, PR/MR).
- Comprendre l'anatomie d'un pipeline d'intégration continue.
- Construire un premier pipeline CI qui teste automatiquement le code.

1.1 Origine et philosophie DevOps

Le **DevOps** est né du divorce historique entre les **développeurs** (qui veulent livrer du changement) et les **opérations** (qui veulent de la stabilité). Ce « mur de la confusion » provoquait des déploiements rares, risqués et conflictuels. Le mouvement DevOps (popularisé vers 2009, *DevOpsDays*) propose de **réunir Dev et Ops** par la culture et l'outillage.

📖 Définition de travail

DevOps n'est **ni un métier, ni un outil** : c'est une **culture** et un ensemble de pratiques visant à raccourcir le cycle entre une idée et sa mise en production, de façon fiable et répétable.

Le modèle CAMS

Pilier	Idée	Exemple concret
C — Culture	Collaboration, responsabilité partagée	Dev et Ops « on call » ensemble
A — Automation	Automatiser ce qui est répétitif	Build, tests, déploiement
M — Measurement	Mesurer pour décider	Lead time, taux d'échec, MTTR
S — Sharing	Partager savoir et feedback	Post-mortems sans blâme

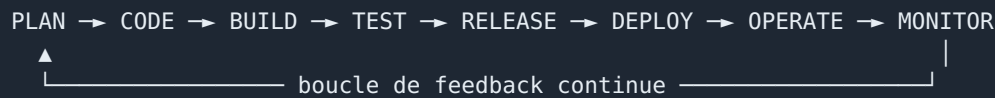
Les métriques DORA (référence du secteur)

1. **Deployment Frequency** — à quelle fréquence on déploie.
2. **Lead Time for Changes** — du commit à la production.
3. **Change Failure Rate** — % de déploiements provoquant un incident.
4. **MTTR** (Mean Time To Restore) — temps de remise en service.

💡 CI ≠ CD ≠ CD

CI = Intégration Continue (on intègre/teste à chaque commit). **CD** = Livraison Continue (l'artefact est toujours *déployable*, déploiement manuel). **CD** = Déploiement Continu (déploiement *automatique* en prod). On gravit ces marches dans cet ordre.

1.2 Le cycle de vie DevOps



- **Plan / Code** : backlog, branches, revue.
- **Build / Test** : compilation, tests automatisés, analyse de qualité/sécurité.
- **Release / Deploy** : artefact versionné, déploiement (manuel ou auto).
- **Operate / Monitor** : exploitation, supervision, alertes → retour au Plan.

1.3 Git & Git Flow (rappels utiles)

Commandes essentielles

```
git init                # initialiser un dépôt
git clone <url>          # cloner
git switch -c feature/paiement # créer + basculer sur une branche
git add . && git commit -m "feat: ajout du paiement"
git push -u origin feature/paiement
# puis ouvrir une Pull Request / Merge Request
```

Convention de messages de commit (Conventional Commits)

```
feat:    nouvelle fonctionnalité
fix:     correction de bug
docs:    documentation
refactor: refonte sans changement fonctionnel
test:    ajout/modif de tests
chore:   tâches techniques (build, deps)
```

Stratégies de branches

Stratégie	Principe	Pour qui
Git Flow	Branches <code>main</code> , <code>develop</code> , <code>feature/*</code> , <code>release/*</code> , <code>hotfix/*</code>	Releases planifiées
GitHub Flow	<code>main</code> + branches courtes + PR + déploiement continu	Web/SaaS, livraison fréquente
Trunk-Based	Commits fréquents sur <code>main</code> , feature flags	Équipes matures, CI forte

⚠ Branches longues = douleur d'intégration

Plus une branche vit longtemps, plus la fusion est risquée. La CI encourage des branches **courtes** et des intégrations **fréquentes**.

1.4 Anatomie d'un pipeline CI

Un pipeline est une suite d'**étapes (stages)** déclenchées par un événement (push, PR). Étapes typiques :

```
push → [checkout] → [install deps] → [lint] → [build] → [tests] → [artefact] → (✓/✗)
```

Principes : **rapide** (feedback en minutes), **reproductible** (mêmes versions partout), **fail fast** (échouer tôt), **vert obligatoire** avant fusion.

1.5 Atelier — Pipeline CI

🔧 Atelier 1 — Durée 2 h — En binômes

Mission : créer un dépôt avec une petite application, écrire un test, et mettre en place un pipeline CI qui s'exécute à chaque push et bloque la fusion si les tests échouent.

Variante A — GitHub Actions (Node.js)

Fichier `.github/workflows/ci.yml` :

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm run lint --if-present
      - run: npm test
```

Test minimal (`sum.test.js` avec Jest) :

```
const sum = (a, b) => a + b;

test('additionne deux nombres', () => {
  expect(sum(2, 3)).toBe(5);
});
```

Variante B — GitLab CI

Fichier `.gitlab-ci.yml` :

```
stages: [build, test]

build:
  stage: build
  image: node:20
  script:
    - npm ci
    - npm run build --if-present

test:
  stage: test
  image: node:20
  script:
    - npm ci
    - npm test
```

Étapes guidées : 1. (20 min) Créer le dépôt, ajouter l'app et le test ; vérifier `npm test` en local. 2. (30 min) Ajouter le fichier de workflow et pousser ; observer l'exécution. 3. (20 min) **Casser** volontairement le test → constater le pipeline rouge. 4. (20 min) Protéger la branche `main` (fusion interdite si CI rouge). 5. (30 min) Restitution + lecture des logs.

 Notes formateur — Atelier 1

Le « aha moment » est l'étape 3 : voir la fusion **bloquée** par un test rouge fait comprendre la valeur de la CI. Montrez aussi le cache des dépendances (gain de temps) et l'exécution sur PR.

Quiz — Jour 1

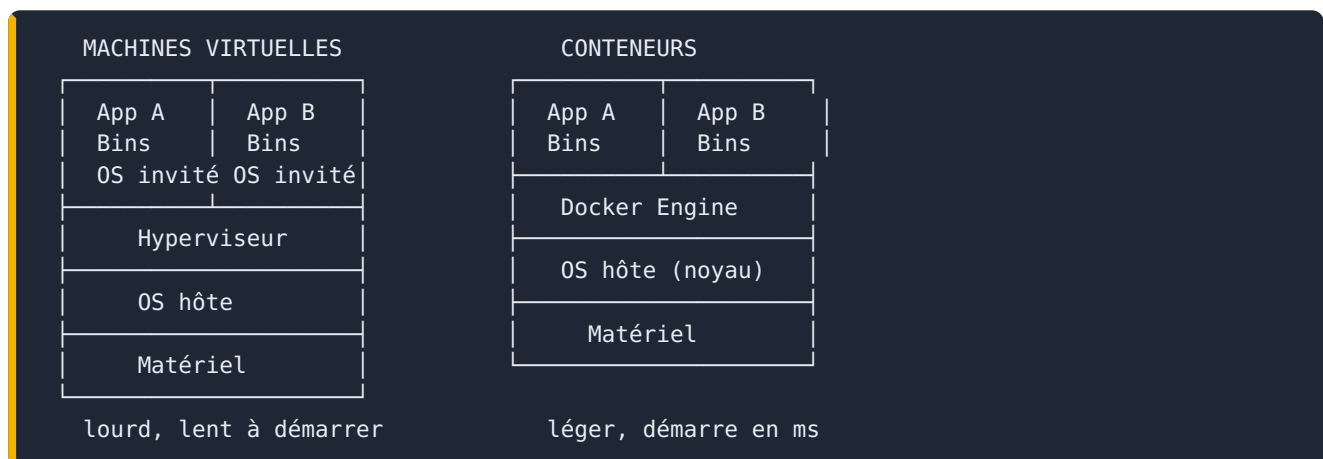
1. Que veut dire CAMS ?
 2. Donnez deux des quatre métriques DORA.
 3. Différence entre Livraison Continue et Déploiement Continu ?
 4. Pourquoi préférer des branches courtes ?
 5. Que fait `git switch -c maBranche` ?
-

Jour 2 — Docker : conteneurisation

🎯 Objectifs du Jour 2

- Expliquer la différence entre conteneur et machine virtuelle.
- Manipuler images, conteneurs, volumes et réseaux Docker.
- Écrire un Dockerfile propre et optimisé (multi-stage).
- Orchestrer plusieurs services en local avec Docker Compose.
- Publier une image sur un registre.

2.1 Conteneurs vs machines virtuelles



Un conteneur **partage le noyau** de l'hôte et n'embarque que l'application + ses dépendances : il est **léger, rapide, portable** (« ça marche chez moi » → « ça marche partout »).

2.2 Concepts Docker

Objet	Définition	Analogie
Image	Modèle en lecture seule (couches)	Le « moule » / une classe
Conteneur	Instance en exécution d'une image	L'objet / le gâteau
Volume	Stockage persistant hors du conteneur	Disque externe
Réseau	Réseau virtuel reliant des conteneurs	Le LAN privé
Registry	Dépôt d'images (Docker Hub, GHCR)	App store d'images

Commandes de base

```
docker run -d -p 8080:80 --name web nginx      # lancer un conteneur en arrière-plan
docker ps                                     # conteneurs actifs
docker logs -f web                            # suivre les logs
docker exec -it web sh                        # ouvrir un shell dedans
docker stop web && docker rm web               # arrêter et supprimer
docker images                                 # lister les images
docker volume create data                     # créer un volume
docker network create app-net                 # créer un réseau
```

2.3 Le Dockerfile

Exemple commenté (application Node.js), **multi-stage** pour une image finale légère :

```
# ----- Étape 1 : build -----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci                                # installe TOUTES les deps (dont dev)
COPY . .
RUN npm run build                          # compile (ex. TypeScript -> dist/)

# ----- Étape 2 : runtime -----
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev                     # uniquement les deps de prod
COPY --from=build /app/dist ./dist
EXPOSE 3000
USER node                                 # ne pas tourner en root
CMD ["node", "dist/server.js"]
```

Bonnes pratiques Dockerfile

- **Image de base minimale** (`-alpine` , `-slim` , ou `distroless`).
- **Ordonner du moins au plus changeant** pour profiter du cache de couches (copier `package.json` avant le code).
- **Multi-stage** pour ne pas embarquer les outils de build.
- `.dockerignore` (exclure `node_modules` , `.git` ...).
- **Ne pas tourner en root**, ne pas stocker de secrets dans l'image.
- **Tag explicite** des versions (éviter `latest` en prod).

```
# .dockerignore
node_modules
.git
*.log
dist
```

! Jamais de secrets dans une image

Les couches d'image sont consultables. N'y copiez **jamais** mots de passe, clés ou tokens. Utilisez des variables d'environnement à l'exécution, ou des secrets gérés (Docker/K8s secrets, coffre-fort).

2.4 Docker Compose

Compose décrit **plusieurs services** dans un fichier et les démarre ensemble. Idéal pour le développement local (app + base de données).

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://app:secret@db:5432/quickbite
    depends_on:
      - db
    networks: [app-net]

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: quickbite
    volumes:
      - db-data:/var/lib/postgresql/data
    networks: [app-net]

volumes:
  db-data:
networks:
  app-net:
```

```
docker compose up -d --build    # construire et démarrer
docker compose ps               # état des services
docker compose logs -f api      # logs d'un service
docker compose down -v          # tout arrêter et supprimer les volumes
```

2.5 Registres d'images

```
# Docker Hub
docker login
docker build -t monuser/quickbite-api:1.0.0 .
docker push monuser/quickbite-api:1.0.0

# GitHub Container Registry (GHCR)
echo $TOKEN | docker login ghcr.io -u monuser --password-stdin
docker tag quickbite-api ghcr.io/monuser/quickbite-api:1.0.0
docker push ghcr.io/monuser/quickbite-api:1.0.0
```

💡 Versionner par tag sémantique

Poussez à la fois un tag précis (`1.0.0`) et un tag mouvant (`1` ou `stable`). En prod, déployez le tag **précis** pour la traçabilité.

2.6 Atelier — Conteneuriser une application

🔧 Atelier 2 — Durée 2 h 30 — En binômes

Mission : conteneuriser la QuickBite API et sa base de données, la lancer via Compose, puis publier l'image sur un registre.

Étapes : 1. (30 min) Écrire le `Dockerfile` (multi-stage) + `.dockerignore`. `docker build` puis `docker run`, tester l'endpoint `GET /health`. 2. (40 min) Ajouter une base PostgreSQL via `docker-compose.yml` ; relier l'API au service `db`. 3. (30 min) Ajouter un **volume** pour persister les données ; vérifier la persistance après `down / up`. 4. (30 min) Tagger et **pousser** l'image sur Docker Hub ou GHCR. 5. (20 min) Restitution : taille de l'image, gain du multi-stage, pièges rencontrés.

Critères de réussite (DoD de l'atelier) :

- `docker compose up` démarre API + DB sans erreur.
- L'API répond et lit/écrit en base.
- L'image finale est < 200 Mo (grâce au multi-stage).
- L'image est visible dans le registre.

💡 Notes formateur — Atelier 2

Pièges fréquents : l'API démarre avant la DB (montrer `depends_on` + une logique de retry / healthcheck), confusion entre port interne et port publié, oubli du `.dockerignore` (image énorme). Faites comparer la taille d'image avec et sans multi-stage : effet pédagogique garanti.

Quiz — Jour 2

1. Conteneur vs VM : quelle est la différence clé ?

2. À quoi sert un volume ?
 3. Pourquoi un build multi-stage ?
 4. Pourquoi copier `package.json` avant le code source ?
 5. Que fait `docker compose down -v` ?
-

Jour 3 — Introduction à Kubernetes

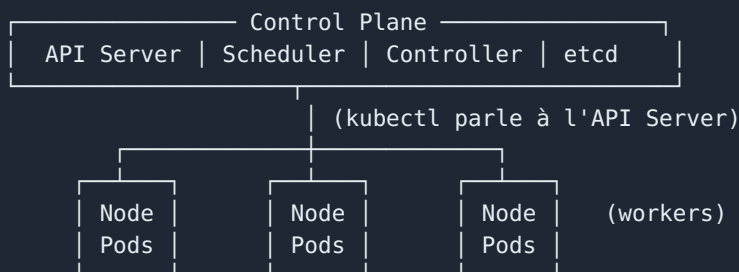
🎯 Objectifs du Jour 3

- Expliquer le rôle d'un orchestrateur de conteneurs.
- Décrire l'architecture de Kubernetes et ses objets de base.
- Déployer une application, l'exposer et la mettre à l'échelle.
- Gérer la configuration (ConfigMap/Secret) et l'isolation (Namespace).
- Comprendre les bases du RBAC et de la sécurité.

3.1 Pourquoi un orchestrateur ?

Docker lance des conteneurs **sur une machine**. En production, on veut : plusieurs machines, du **redémarrage automatique**, de la **mise à l'échelle**, des **mise à jour sans coupure**, de la **répartition de charge**, de l'**auto-réparation**. C'est le rôle de **Kubernetes (K8s)**.

3.2 Architecture et objets



Objet	Rôle
Pod	Plus petite unité déployable : un (ou +) conteneurs partageant réseau/stockage
ReplicaSet	Maintient <i>N</i> copies d'un Pod
Deployment	Gère les ReplicaSet : mises à jour progressives, rollback
Service	Adresse réseau stable + load balancing vers des Pods
Ingress	Routage HTTP(S) externe vers des Services (noms de domaine, chemins)

Un Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quickbite-api
spec:
  replicas: 3
  selector:
    matchLabels: { app: quickbite-api }
  template:
    metadata:
      labels: { app: quickbite-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/monuser/quickbite-api:1.0.0
          ports:
            - containerPort: 3000
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "256Mi" }
```

Un Service

```
apiVersion: v1
kind: Service
metadata:
  name: quickbite-api
spec:
  selector: { app: quickbite-api }
  ports:
    - port: 80
      targetPort: 3000
  type: ClusterIP      # interne ; NodePort/LoadBalancer pour exposer
```

Un Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: quickbite-ingress
spec:
  rules:
    - host: quickbite.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: quickbite-api
                port: { number: 80 }
```

Commandes kubectl essentielles

```
kubectl apply -f deployment.yaml      # créer/mettre à jour depuis un manifeste
kubectl get pods,svc,deploy           # lister les objets
kubectl describe pod <nom>            # détails + événements (debug)
kubectl logs -f <pod>                 # logs
kubectl scale deploy/quickbite-api --replicas=5
kubectl rollout status deploy/quickbite-api
kubectl rollout undo deploy/quickbite-api # rollback
kubectl port-forward svc/quickbite-api 8080:80 # accès local rapide
```

■ Déclaratif, pas impératif

On décrit **l'état souhaité** dans des YAML, et Kubernetes **converge** en continu vers cet état (auto-réparation). C'est la base de la philosophie GitOps vue au Jour 4.

3.3 Configuration : ConfigMaps, Secrets, Namespaces

```
apiVersion: v1
kind: ConfigMap
metadata: { name: quickbite-config }
data:
  APP_MODE: "production"
  PAGE_SIZE: "20"
  ---
apiVersion: v1
kind: Secret
metadata: { name: quickbite-secret }
type: Opaque
stringData:
  DATABASE_PASSWORD: "secret-a-changer"
```

Injection dans un conteneur :


```
envFrom:
- configMapRef: { name: quickbite-config }
- secretRef:    { name: quickbite-secret }
```

⚠ Les Secrets K8s sont encodés, pas chiffrés

Par défaut un Secret est seulement encodé en base64. Activez le chiffrement *at rest* de etcd et/ou utilisez un gestionnaire externe (Sealed Secrets, External Secrets, Vault). Ne commitez jamais un Secret en clair.

Namespaces : cloisonnent les ressources (ex. `dev`, `staging`, `prod`) et servent de périmètre aux quotas et au RBAC.

```
kubectl create namespace dev
kubectl apply -f deployment.yaml -n dev
```

3.4 RBAC & sécurité

Le **RBAC** (Role-Based Access Control) définit *qui* peut faire *quoi* sur *quelles* ressources.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: dev, name: lecteur-pods }
rules:
- apiGroups: ["" ]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { namespace: dev, name: bind-lecteur }
subjects:
- kind: User
  name: alice
roleRef:
  kind: Role
  name: lecteur-pods
  apiGroup: rbac.authorization.k8s.io
```

Bonnes pratiques sécurité : **moindre privilège**, conteneurs **non-root**, `resources.limits`, **NetworkPolicies**, scan d'images, mises à jour régulières.

3.5 Atelier — Déployer sur Minikube / K3s

🔧 Atelier 3 — Durée 2 h 30 — En binômes

Mission : déployer la QuickBite API sur un cluster local, l'exposer via Ingress, la configurer par ConfigMap/Secret, puis la mettre à l'échelle et tester un rollback.

Étapes : 1. (15 min) Démarrer le cluster : `minikube start` (activer l'ingress : `minikube addons enable ingress`). 2. (30 min) Appliquer `Deployment` + `Service` ; vérifier les Pods (`kubectl get pods`). 3. (25 min) Ajouter `ConfigMap` + `Secret` et les injecter. 4. (25 min) Exposer via `Ingress` ; ajouter l'hôte dans `/etc/hosts` (`minikube ip`), tester dans le navigateur. 5. (25 min) `kubectl scale` à 5 réplicas ; déployer une **mauvaise** image puis `rollout undo` . 6. (30 min) Restitution + debug (`describe` , `logs`).

Critères de réussite :

- 3 Pods `Running` et `Ready` .
- L'application répond via l'Ingress.
- Le rollback restaure la version saine.

Notes formateur — Atelier 3

Anticipez : image absente du cluster (utiliser `minikube image load` ou un registre), Ingress non prêt (addon), confusion Service/Ingress. Insistez sur `kubectl describe` + section `Events` comme premier réflexe de debug.

Quiz — Jour 3

1. Qu'est-ce qu'un Pod ?
2. Rôle d'un Deployment vs d'un Service ?
3. Différence ConfigMap / Secret ?
4. Que fait `kubectl rollout undo` ?
5. Que signifie « approche déclarative » ?

Jour 4 — Automation & DevOps avancé

🎯 Objectifs du Jour 4

- Assembler une chaîne CI/CD complète : build → test → image → déploiement.
- Packager une application Kubernetes avec Helm.
- Mettre en place une observabilité de base (Prometheus + Grafana).
- Comprendre et appliquer le GitOps avec Argo CD.

4.1 Pipeline CI/CD de bout en bout

Objectif : à chaque tag, **construire l'image, la pousser, et déployer** sur Kubernetes.

```
name: CI-CD
on:
  push:
    tags: [ 'v*' ]
jobs:
  build-push:
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${ github.repository }:${ github.ref_name }
  deploy:
    needs: build-push
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Déployer sur Kubernetes
        run: |
          kubectl set image deploy/quickbite-api \
            api=ghcr.io/${ github.repository }:${ github.ref_name } \
            --namespace=prod
```

💡 Sécuriser les secrets de pipeline

N'écrivez jamais un token en clair : utilisez les **secrets** du dépôt (`${ secrets.X }`) et le principe du moindre privilège pour le compte de déploiement.

4.2 Helm Charts

Helm est le « gestionnaire de paquets » de Kubernetes : il *templatisé* les manifestes et les paramètre par environnement.

```
quickbite-chart/  
├─ Chart.yaml          # métadonnées du chart  
├─ values.yaml         # valeurs par défaut  
└─ templates/  
    ├─ deployment.yaml # avec des {{ .Values.* }}  
    ├─ service.yaml  
    └─ ingress.yaml
```

`values.yaml` :

```
replicaCount: 3  
image:  
  repository: ghcr.io/monuser/quickbite-api  
  tag: "1.0.0"  
service:  
  port: 80
```

Extrait de `templates/deployment.yaml` :

```
spec:  
  replicas: {{ .Values.replicaCount }}  
  template:  
    spec:  
      containers:  
        - name: api  
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

```
helm install quickbite ./quickbite-chart -n prod  
helm upgrade quickbite ./quickbite-chart --set image.tag=1.1.0 -n prod  
helm rollback quickbite 1 -n prod  
helm uninstall quickbite -n prod
```

Pourquoi Helm ?

Sans Helm, on duplique des YAML quasi identiques par environnement. Helm **factorise** (un template, plusieurs `values`) et fournit versioning + rollback applicatif.

4.3 Observabilité : Prometheus & Grafana

Les **3 piliers** : *Logs* (que s'est-il passé), *Métriques* (combien/à quelle vitesse), *Traces* (parcours d'une requête).

- **Prometheus** : collecte des **métriques** en *scrappant* des endpoints `/metrics` , les stocke en séries temporelles, et alerte (Alertmanager).
- **Grafana** : **visualise** ces métriques en tableaux de bord.

```

Application(/metrics) —scrape→ Prometheus —requête PromQL→ Grafana (dashboards)
                                   |
                                   → Alertmanager → email / Slack

```

Exemple de requête PromQL (taux d'erreurs HTTP 5xx) :

```

sum(rate(http_requests_total{status=~"5.."}[5m]))
/ sum(rate(http_requests_total[5m]))

```

💡 Les 4 signaux dorés (Google SRE)

À superviser en priorité : **Latence, Trafic, Erreurs, Saturation**. Un dashboard qui couvre ces quatre signaux suffit souvent à détecter 90 % des incidents.

4.4 GitOps avec Argo CD

GitOps : l'état souhaité du cluster vit dans **Git** (source de vérité unique). Un agent (**Argo CD**) compare en continu Git ↔ cluster et **synchronise** automatiquement.

```

Développeur → commit/PR → Dépôt Git (manifestes/Helm)
                                   |
                                   Argo CD surveille
                                   ▼
Cluster Kubernetes (auto-sync vers l'état Git)

```

Avantages : **traçabilité** (tout passe par Git), **rollback** = `git revert`, **auditabilité**, fin des `kubectl apply` manuels en prod.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: quickbite, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/monuser/quickbite-gitops
    targetRevision: main
    path: k8s/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated: { prune: true, selfHeal: true }

```

4.5 Atelier final — Chaîne complète CI/CD → Docker → Kubernetes

Atelier final — Durée 3 h — En équipes

Mission : assembler tout le module en une chaîne de bout en bout, du commit à la production locale.

Étapes : 1. (30 min) Le pipeline CI **teste** puis **construit et pousse** l'image (Jour 1 + Jour 2). 2. (40 min) Packager l'app en **Helm chart** paramétré (Jour 4). 3. (40 min) Déployer sur le cluster local via Helm ; exposer via Ingress (Jour 3). 4. (40 min) Brancher **Prometheus + Grafana** et afficher un dashboard (latence/erreurs). 5. (30 min, *bonus*) Mettre l'app sous **Argo CD** : un commit dans le repo GitOps déclenche le déploiement. 6. (20 min) Démonstration par équipe : « du commit à la prod ».

Critères de réussite (DoD) :

- Un push déclenche tests + build + push d'image (CI verte).
- `helm upgrade` déploie la nouvelle version sans coupure.
- Le dashboard Grafana affiche au moins un signal doré.
- (Bonus) Argo CD montre l'application *Synced / Healthy*.

Notes formateur — Atelier final

Constituez des équipes mixtes (profils dev + ops). Timeboxez fermement chaque étape ; le bonus Argo CD n'est abordé que si le reste fonctionne. Gardez 20 min pleines pour les démos : raconter « du commit à la prod » ancre toute la semaine.

Évaluation finale du Module 2

Partie A — QCM

1. CAMS signifie : **Culture, Automation, Measurement, Sharing.**
2. CI vs CD : ?
3. Multi-stage Docker sert à : **réduire la taille de l'image finale.**
4. Un Deployment gère : **les ReplicaSet / les mises à jour et le rollback.**
5. Helm sert à : **packager/paramétrer des manifestes Kubernetes.**
6. GitOps : la source de vérité est : **Git.**
7. Prometheus collecte des : **métriques.**
8. Les 4 signaux dorés : **latence, trafic, erreurs, saturation.**

Partie B — Étude de cas (atelier noté)

On évalue l'atelier final sur la grille suivante :

Critère	Points
Pipeline CI fonctionnel (tests bloquants)	4
Image conteneurisée propre (multi-stage, sécurité)	4
Déploiement Kubernetes via Helm	5
Exposition + configuration (Ingress, ConfigMap/Secret)	3
Observabilité (dashboard)	2
Bonus GitOps (Argo CD)	+2

✓ Clôture du Module 2

Reliez explicitement au Module 1 : la chaîne DevOps n'a de valeur que si elle sert un **flux de valeur** agile (livrer tôt et souvent). Annoncez le Module 3 : sécuriser l'application que l'on vient d'apprendre à livrer.